Understanding Agentic AI

A plain-language guide to the model, tools, memory and the loop

1. The Core Idea

A normal chatbot answers once: you ask, it replies, done.

An **agent** is an AI that works in a **loop** to reach a goal:

- **Think:** look at the goal and what has happened so far
- **Act:** do something (search the web, run code, read a file, send an email)
- **Observe:** see the result of that action
- **Repeat** until the goal is done

> That is it. Everything else in agentic AI is detail built on top of this loop.

A simple example

"Find the cheapest flight to Delhi next Friday and put it in my calendar."

- Think: I need flight prices first.
- Act: call a flight search tool.
- Observe: results come back.
- Think: the cheapest is at 6 AM. Now I need to add it to the calendar.
- Act: call the calendar tool.
- Observe: success. Done.

A chatbot could only *tell you how* to do this. An agent actually does it.

The four parts

Part	Role	One-liner
Model	The brain (an LLM like Claude)	Decides
Tools	The hands (functions it can call)	Do
Memory	The notebook	Remembers
Loop	The manager (regular code)	Keeps it running

Why it feels confusing

Videos often mix up levels: some explain the concept, some show a framework (LangChain, CrewAI, etc.), some show multi-agent systems. All of those are *built on* the simple loop above. If the loop is not clear first, the rest sounds like noise.

2. The Model (the brain)

What it is: The LLM (Claude, GPT, Gemini, etc.). It is the only part that "understands" language and makes decisions.

What it actually does: It takes in text (the goal, the history, the list of available tools) and produces text as output. That is all it can do. It cannot browse the web or send emails by itself.

The key trick: Instead of only replying with a normal answer, the model is allowed to reply with a *request to use a tool*. For example:

```
Call tool: search_flights
Arguments: { "to": "Delhi", "date": "2026-11-06" }
```

This is not the model *doing* the search. It is the model *asking* for the search to be done. The model decides *what* should happen. Something else makes it happen.

What the model is good at here

- Breaking a goal into steps
- Choosing which tool fits the current step
- Reading results and deciding what to do next
- Noticing when something went wrong and trying another approach

Its limitation: It has no memory of its own between calls. Every time you call it, you must send the whole relevant conversation again. This is why memory matters.

3. Tools (the hands)

What they are: Ordinary functions written by a developer, such as `search_flights()`, `add_calendar_event()`, `send_email()`, `query_database()`, `run_python_code()`.

How the model knows about them: The developer gives the model a *description* of each tool: its name, what it does, and what inputs it needs.

```
Tool: search_flights
Description: Finds flights between two cities on a date
Inputs: from (text), to (text), date (YYYY-MM-DD)
```

The model reads these descriptions like a menu and picks what fits.

What happens when the model picks one

- 1. The model outputs a tool request.
- 2. Your code sees that request and **actually runs** the real function.
- 3. The result (say, a list of flights) is sent back to the model as text.
- 4. The model reads it and decides the next step.

Common types of tools

- **Information tools:** web search, database lookup, reading files
- **Action tools:** sending email, booking, updating a calendar, writing files
- **Computation tools:** calculator, running code
- **Other agents:** one agent can be a tool for another

Why tools matter: Without them, the model can only talk. With them, it can affect the real world. Tools also fix the model's weaknesses: it is unreliable at exact math, so you give it a calculator; it does not know today's prices, so you give it search.

4. Memory (the notebook)

Since the model forgets everything between calls, memory is how an agent "remembers." There are two main kinds.

Short-term memory (within one task)

This is simply the running transcript: the goal, every action taken, and every result received. Each time the loop runs, this transcript is sent back to the model so it knows what has already happened.

In our example, when the agent goes to add the calendar event, it still knows the cheapest flight was at 6 AM *because that result is in the transcript*.

Limit: The model can only read so much text at once (its "context window"). Long tasks can overflow it, so developers often summarize or trim older parts.

Long-term memory (across sessions)

Information stored outside the model, usually in a database or files, and loaded back in when relevant. Examples:

- "User prefers window seats and morning flights"
- "User's home airport is Hyderabad"
- Notes from last week's task

Often this is done by saving notes, then *searching* them later and pasting the relevant ones into the prompt. Terms like "vector database" and "RAG" are just ways to search stored memory by meaning.

> **Simple way to think of it:** Short-term memory is the notepad on your desk for today's job. Long-term memory is the filing cabinet you check when something from the past is relevant.

5. The Loop / Orchestration (the manager)

What it is: Regular code (Python, JavaScript, etc.), not AI. It is the glue that connects the other three parts and keeps things running.

What it does, step by step

- 1. Start with the user's goal
- 2. Send the goal, history, and tool list to the model
- 3. Read the model's response: if it is a **tool request**, run that tool, add the result to the history, and go back to step 2; if it is a **final answer**, stop and show it to the user
- 4. Enforce safety limits (for example, stop after 20 steps so it cannot loop forever)

Simplified pseudo-code

```
history = [user_goal]

while True:
    response = model(history, tools)

    if response.is_tool_request:
```

```
        result = run_tool(response.tool, response.arguments)
        history.append(result)
    else:
        print(response.answer)
        break
```

That small loop *is* the core of an agent.

What else the manager handles

- **Error handling:** if a tool fails, pass the error back so the model can try something else
- **Permissions:** asking the user to approve risky actions (like sending money or deleting files)
- **Limits:** maximum steps, time, or cost
- **Logging:** recording what happened for debugging

6. How All Four Work Together

The flight example, with each part labeled:

Step	Who does it	What happens
1	Loop	Sends goal + tool list to the model
2	Model	Replies: "call search_flights for Delhi, next Friday"
3	Loop	Sees the request and runs the real function
4	Tool	Returns a list of flights
5	Memory	Result is added to the transcript
6	Loop to Model	Sends updated transcript back
7	Model	Picks the cheapest, replies: "call add_calendar_event"
8	Loop to Tool	Runs it, event is created
9	Model	Sees success, replies with the final answer: "Done! 6 AM flight added to your calendar."
10	Loop	Sees a final answer, stops

One-line summary

- **Model** decides.
- **Tools** do.
- **Memory** remembers.
- **Loop** keeps it all running until the job is finished.